Understanding Inheritance in Python

A comparison of duplicated code versus class inheritance, with worked examples in Python.

Inheritance is one of the four core principles of Object-Oriented Programming (OOP), alongside encapsulation, polymorphism, and abstraction. It allows one class — referred to as the **child** or **derived** class — to reuse the attributes and methods defined in another class, referred to as the **parent** or **base** class.

Rather than reimplementing shared logic in every class, the child class inherits it directly from the parent, reducing repetition and centralizing maintenance.

1. Code Without Inheritance

Consider two independent classes, `Dog` and `Cat`, each implementing its own constructor and methods.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Bark")

    def eat(self):
        print(f"{self.name} is eating")

    def sleep(self):
        print(f"{self.name} is sleeping")


class Cat:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Meow")

    def eat(self):
        print(f"{self.name} is eating")

    def sleep(self):
        print(f"{self.name} is sleeping")


dog = Dog("Tommy")
cat = Cat("Kitty")
```

```
dog.eat()
dog.speak()
dog.sleep()

cat.eat()
cat.speak()
cat.sleep()
```

OUTPUT

```
  Tommy is eating
  Bark
  Tommy is sleeping
  Kitty is eating
  Meow
  Kitty is sleeping
```

2. The Problem: Code Duplication

Although the program above runs correctly, both classes define an identical constructor, an identical `eat()` method, and an identical `sleep()` method:

```
def __init__(self, name):
    self.name = name

def eat(self):
    print(f"{self.name} is eating")

def sleep(self):
    print(f"{self.name} is sleeping")
```

In short, the same code was written twice. Now imagine having to model several more animals — `Dog`, `Cat`, `Lion`, `Tiger`, `Horse`, and `Elephant`. The same constructor, `eat()` method, and `sleep()` method would need to be repeated in every single one of them. This pattern, where identical code is rewritten across multiple classes, is known as **code duplication**, and it directly increases the surface area for bugs and long-term maintenance effort.

3. Refactoring With Inheritance

The shared logic can be extracted into a single parent class, `Animal`:

```
class Animal:
    def __init__(self, name):
        self.name = name
```

```
    def eat(self):
        print(f"{self.name} is eating")

    def sleep(self):
        print(f"{self.name} is sleeping")
```

`Dog` and `Cat` then inherit from `Animal` and only define what makes them distinct:

```
class Dog(Animal):
    def speak(self):
        print("Bark")


class Cat(Animal):
    def speak(self):
        print("Meow")
```

The usage code remains unchanged:

```
dog = Dog("Tommy")
cat = Cat("Kitty")

dog.eat()
dog.speak()
dog.sleep()

cat.eat()
cat.speak()
cat.sleep()
```

OUTPUT

```
Tommy is eating
Bark
Tommy is sleeping
Kitty is eating
Meow
Kitty is sleeping
```

The output is identical to the non-inherited version, but the `eat()` method, the `sleep()` method, and the constructor now exist in exactly one place — on `Animal` — and both `Dog` and `Cat` get them for free.

4. How Python Resolves Methods

When Python encounters `class Dog(Animal):`, it establishes that `Dog` inherits every attribute and method defined on `Animal`. As a result, `Dog` automatically has access to `__init__()` and `eat()` without redefining them, and only needs to add its own `speak()` method.

Method lookup follows a defined search order, generally referred to as the **Method Resolution Order (MRO)**. Given the instance below:

```
dog = Dog("Tommy")
dog.eat()
```

Python evaluates the call as follows:

- Does the `Dog` class define `eat()`? — No.
- Does the parent class, `Animal`, define it? — Yes.
- `Animal.eat()` is executed.

Conversely, calling `dog.speak()` resolves immediately at the `Dog` level, since `Dog` defines its own `speak()` method. Python never has to consult `Animal` in that case, because the search stops as soon as a match is found.

5. Structural Comparison

The diagrams below summarize how methods are organized in memory under each approach.

Without Inheritance

`Dog` → __init__(), eat(), sleep(), speak()
`Cat` → __init__(), eat(), sleep(), speak()

Note: __init__(), eat(), and sleep() are duplicated in both classes.

With Inheritance

`Animal` → __init__(), eat(), sleep()
→ `Dog` → speak()
→ `Cat` → speak()

Note: Only one copy of __init__(), eat(), and sleep() exists, shared by both subclasses.

6. A Second Example: Student and Teacher

The same pattern applies beyond animals. Consider two unrelated classes, `Student` and `Teacher`, that duplicate the same constructor and display method:

```
class Student:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)


class Teacher:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)
```

Introducing a common `Person` base class removes the duplication entirely:

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)


class Student(Person):
    pass


class Teacher(Person):
    pass


s = Student("Alice")
t = Teacher("Bob")

s.show()
t.show()
```

OUTPUT

```
Alice
Bob
```

Even though `Student` and `Teacher` contain no methods of their own, both instances behave correctly because they inherit everything from `Person`.

The Role of pass

The keyword `pass` is a syntactic placeholder indicating that a class body is intentionally empty:

```
class Student(Person):
    pass
```

It signals that the class does not add any new behavior at this time, while still inheriting the full set of methods and attributes from its parent.

7. Summary Comparison

Without Inheritance	With Inheritance
Same code written multiple times	Common code written once
More lines of code overall	Fewer lines of code overall
Harder to maintain	Easier to maintain
Changes must be repeated in every class	Changes are made once, in the parent class
Higher likelihood of inconsistent bugs	Lower likelihood of inconsistent bugs

8. A Real-World Analogy

The relationship between a parent and child class mirrors inheritance within a family. If a parent has brown eyes and black hair, their children inherit those same traits by default, while still being free to develop characteristics of their own — a son who plays football, or a daughter who plays piano.

The equivalent structure in Python:

```
class Animal:
    def eat(self):
        print("Eating")


class Dog(Animal):
    def bark(self):
        print("Barking")
```

- `Dog` inherits the `eat()` method from `Animal`.
- `Dog` additionally defines its own `bark()` method.

Key takeaway. Inheritance lets a child class reuse the attributes and methods of a parent class. This reduces code duplication, simplifies long-term maintenance, and models real-world “is-a” relationships — for example, a `Dog` is an `Animal`. When a method is called on an object, Python first checks the object's own class; if the method is not found there, Python searches the parent class or classes in order.